LAST YEAR PROJECT

Complete Product Vision, Scope, Architecture & Development Brief

Product category: AI-powered E-commerce Business Intelligence & R&D SaaS

A handoff document for an AI coding assistant or technical/product partner to understand the idea, make sound implementation decisions, and build it incrementally.

Core vision
Connect existing e-commerce websites, unify their business data, explain performance and risk, assist customers, analyze marketing outcomes, and act as an AI research and development partner for small and medium businesses.

Current scope intentionally excludes a native website builder, landing-page builder, and hosted page-publishing platform. The product focuses on existing websites and the AI R&D workflow.

1. Executive Summary

Last Year Project is a multi-tenant SaaS platform for e-commerce businesses. It connects to existing websites built with different technologies and turns their products, orders, customer-service interactions, marketing data, and security signals into a unified business intelligence layer.

The platform should not merely show charts. It should help a merchant understand what happened, why it may have happened, what should be investigated next, and which action or experiment could improve results. The signature capability is the AI R&D Lab, designed to give smaller businesses access to structured, evidence-based product, pricing, quality, market, and operational research.

The product should be developed incrementally, starting with a reliable Laravel connector and secure data foundation. AI output must be evidence-aware and clearly distinguish facts, hypotheses, confidence, and missing data.

Product goals

- Connect and manage multiple existing e-commerce websites from one SaaS dashboard.
- Normalize website data into a consistent model for products, orders, customers, employees, and status history.
- Provide AI-assisted customer support and order assistance through a website chatbot.
- Report sales, cancellation, delivery, employee, marketing, and estimated profitability metrics.
- Detect suspicious application activity and help manage security risks through layered controls.
- Use an AI R&D; Lab to identify risks and opportunities, propose research, and track experiment outcomes.
- Build a secure, modular foundation that can later support more connectors, channels, and AI capabilities.

Target users

- Small and medium e-commerce merchants that cannot afford separate analysts or an R&D; team.
- Growing online brands managing staff, orders, couriers, and paid advertising.
- Agencies or operators managing multiple client stores, once permissions and client workspaces are mature.

2. Scope Decisions

Included in current product scope

- Multi-tenant SaaS foundation, dashboard, roles, permissions, audit trail, and website registration.
- External website integration, starting with Laravel; later WordPress/WooCommerce and generic REST APIs.
- AI chatbot and controlled sales/order assistance.
- Business analytics, order funnel, employee performance, product and courier outcomes.
- Marketing intelligence, initially with selected advertising integrations and carefully qualified attribution.
- Security/WAF integration, application-level threat signals, suspicious activity monitoring, and fraud-pattern analysis.
- AI R&D; Lab: business risks, product improvement, pricing and margin research, quality insights, market opportunities, experiments, and validation.

Explicitly deferred / out of scope for now

- Native drag-and-drop website builder.
- Single-product landing-page builder.
- Hosted website/page publishing platform and template marketplace.
- A new native commerce engine replacing existing store platforms.
- Unrestricted autonomous AI actions that alter prices, budgets, orders, or security settings without controls.

The product will integrate with existing websites rather than build the websites itself. Future builder work can be reconsidered after the core SaaS has been validated.

3. Functional Modules

3.1 Core SaaS & Website Integration

- Tenant and user management with role-based access.
- Register a website, normalize its domain, verify ownership where possible, and manage integration health.
- Issue, rotate, scope, expire, and revoke machine credentials securely.
- Sync products and orders; ingest signed events/webhooks; support retries, idempotency, and sync diagnostics.
- Keep connector permissions narrow and website-specific.

3.2 AI Chatbot & Sales Assistant

- Answer product, price, availability, delivery, return-policy, and other approved knowledge questions.
- Support Bangla, Banglish, and English where feasible.
- Collect leads only with appropriate consent and provide human handoff.
- Assist with order creation only through validated server-side APIs; verify product availability, prices, address, delivery options, and customer confirmation.
- Track conversations, unresolved questions, leads, assisted orders, and conversion outcomes.

3.3 Business Analytics & Reporting

- Sales, order count, average order value, cancellation, return, delivery, and repeat-purchase metrics.
- Order lifecycle and status history: received, confirmed, cancelled, shipped, delivered, returned, and other source-specific states.
- Employee-wise assignment, confirmation, cancellation, response time, and workload context.
- Product performance, stock alerts, courier performance, and data-quality indicators.
- Estimated contribution profit when cost inputs are available; do not equate revenue or ad-platform ROAS with profit.
- Natural-language business questions with metric definitions, date ranges, evidence, and links to underlying reports.

3.4 Marketing Intelligence

- Integrate selected ad platforms through their approved APIs and account authorization flows.
- Report spend, impressions, clicks, platform-reported conversions, website orders, CPA, ROAS, and estimated contribution profit where data permits.
- Compare campaign results with confirmed, delivered, cancelled, or returned orders when reliable matching is possible.
- Show attribution windows, matching coverage, data freshness, and uncertainty; never imply perfect attribution.
- Recommend investigations and tests before automatically changing campaign budgets.

3.5 Security & Fraud Intelligence

- Cloudflare WAF integration and application-level security monitoring.
- Detect scanner patterns, brute-force attempts, API abuse, unusual request rates, and suspicious events.
- Use risk-based throttling/mitigation with logs, audit history, and false-positive review.
- Analyze duplicate orders and suspicious order patterns separately from website attack detection.
- Escape untrusted input in logs and admin screens; never render raw payloads as HTML.
- Do not treat a path such as /wp-admin, a .php URL, or VPS/emulator traffic alone as proof of malicious intent.

3.6 AI R&D; Lab — Signature Differentiator

The R&D Lab should work like a structured research analyst, not a generic prompt that produces unverified advice. It should combine validated business metrics, statistical/rule-based detection, AI interpretation, evidence, experiments, and outcome tracking.

- Business risk intelligence: falling sales, rising cancellation/return, margin compression, inventory risk, channel dependency, customer complaints, and operational bottlenecks.

- Product research: demand trends, variant performance, recurring customer feedback, product-description mismatch, possible new variants, bundles, improvement, or discontinuation.
- Pricing and profitability: cost-aware margin calculations, discount impact, price-test proposals, competitor comparisons when reliable public data is available.
- Quality and customer experience: complaint themes, delivery/packaging problems, refund patterns, and issues requiring physical inspection or testing.
- Market and competitor research: sourced, dated public information, market-gap hypotheses, and clearly labelled uncertainty.
- Experiment center: hypothesis, test design, sample/time requirements, success metric, baseline, owner approval, result, and lessons learned.

Required fields for every R&D; finding

Field	Purpose
Finding	Problem or opportunity detected.
Evidence	Metrics, data sources, time window, and relevant comparison.
Risk / priority	Business urgency and potential impact.
Confidence	Strength of evidence; distinct from severity.
Hypotheses	Possible causes, explicitly not asserted as facts.
Missing data	Information needed to improve the diagnosis.
Recommended action	What to do next and why.
Experiment plan	How to test the recommendation safely.
Success metric	How improvement will be measured.
Status and outcome	New, reviewing, approved, testing, validated, or rejected; record the actual result.

The system must not invent causes or forecast certainty. If data is inadequate, it should say so and recommend what to measure or inspect next.

4. Technical Architecture

Layer	Proposed technology / responsibility
Main backend	Laravel — system of record, tenant authorization, integrations, order/product normalization, APIs, audit, and business rules.
Dashboard	Blade + Livewire.
AI service	Python + FastAPI for RAG/chat orchestration, R&D; analysis, and later advanced modelling.
Database	PostgreSQL; pgvector may be added for suitable knowledge-retrieval use cases.
Queue/cache	Redis for queues, caching, throttling, and background work.
Deployment	Docker-based local and production deployment.
Edge security	Cloudflare DNS/proxy/TLS/WAF, configured according to plan and tested for false positives.
Website connectors	Versioned APIs, signed machine requests, scoped credentials, webhooks/events, retries, and idempotency.

Architectural rules

- Use a modular monolith for the Laravel core initially; keep the Python AI service as a separately deployable service only where it provides clear value.
- Laravel remains the authoritative source for tenants, websites, orders, permissions, and audit events.

- Do not trust a tenant ID, website ID, role, price, or order status merely because it appears in a browser request.
- All cross-service and connector contracts must be versioned and documented.
- Use background jobs for sync, analytics aggregation, external API calls, and long-running AI work.
- Keep customer PII out of vector indexes by default; define consent, retention, deletion, and access policies.
- Do not store plaintext connector secrets. Show newly generated secrets only once; hash or otherwise securely manage credentials according to the chosen authentication design.

5. Integration & API Direction

A proposed API namespace is /api/v1. Exact endpoints and schemas must be finalized during implementation; the following are functional groups, not a claim that they are already implemented.

API group	Examples of responsibilities
Website management	Register/list/show websites, issue/revoke credentials, health, trigger sync.
Connector ingestion	Heartbeat, signed events, product batches, order batches, sync status.
Chat	Create public sessions, send messages, list conversations, controlled order actions.
Analytics	Overview, orders, employee performance, products, courier outcomes.
Marketing	Ad accounts, campaigns, daily performance, attribution/matching quality.
Security	Security events, rules, incidents, audit history.
R&D;	Findings, recommendations, experiment plans, approval, feedback, and outcome reports.

Connector authentication requirements

- Use TLS and website-bound credentials with narrow scopes, expiry, rotation, and revocation.
- Prefer signed requests using a timestamp, nonce, and body hash with HMAC-SHA256 or another reviewed scheme.
- Reject stale timestamps and replayed nonces; compare signatures safely and rate-limit endpoints.
- Make event ingestion idempotent with unique source event IDs and retry-safe processing.
- Never expose privileged connector secrets to public browser JavaScript.

6. Data Model — Initial Entities

The data model should evolve from real connector requirements. A first-pass schema can include the following logical entities:

Domain	Entities / key concepts
Tenancy & access	tenants, users, tenant_user_roles, websites, website_integrations, api_credentials, audit_logs
Catalog & orders	products, customers, orders, order_items, order_status_history, employees, order_assignments
Integration reliability	integration_events, sync_jobs, webhook_delivery_logs
Chatbot	chatbots, knowledge_sources, knowledge_documents, conversations, chat_messages, chatbot_leads, ai_action_logs
Security	security_events, security_rules, blocked_entities / risk controls
Marketing	ad_connections, ad_accounts, campaigns, ad_performance_daily, attribution/matching records
Analytics & R&D;	analytics_snapshots, ai_recommendations, research_findings, experiments, experiment_results

Use UUIDs consistently if chosen for primary keys and foreign keys. Enforce uniqueness for external IDs per website, retain order-time item/price snapshots, and preserve status history rather than overwriting all

history. Tenant isolation must be enforced by application authorization; PostgreSQL Row-Level Security can provide additional defence but is not a replacement for authorization.

7. Security, Privacy & AI Governance

- Dashboard authentication should use secure sessions/Sanctum as appropriate, CSRF protection, login throttling, session rotation, and role-based authorization.
- Use policies for every tenant-owned resource; test both same-tenant access and cross-tenant denial.
- Log sensitive actions without logging passwords, secrets, session cookies, authorization headers, or unnecessary raw customer payloads.
- Public chatbot access should use short-lived, domain-restricted widget tokens, rate limits, input-size limits, and server-side validation.
- AI must use approved, tenant-scoped data; prevent cross-tenant retrieval and prompt-injection-driven tool misuse.
- Require approval for high-impact actions such as price changes, campaign-budget changes, order cancellation, exports, or security blocks.
- Use staged WAF rollout and test false positives. A token-only integration cannot protect all incoming traffic on an external site unless traffic routing or a supported site-side agent/plugin provides the required enforcement.

AI output policy

- Separate verified facts, computed metrics, hypotheses, recommendations, and forecasts.
- Show data source and analysis window for important findings.
- Give confidence and uncertainty; do not manufacture missing data.
- Use deterministic calculations for money and core metrics; use LLMs for explanation and research orchestration, not as the source of truth for arithmetic.
- Keep a record of recommendation, owner decision, experiment, and measured outcome.

8. Delivery Roadmap

Phase	Primary deliverables	Acceptance focus
1. Foundation	Laravel bootstrap, Docker, PostgreSQL, Redis, auth, tenant model, website registration, credential lifecycle, first Laravel connector.	Clean setup instructions; tenant isolation; revoked credentials rejected; secrets never returned or logged.
2. Data & analytics	Product/order sync, idempotent events, order-status history, employee and product reporting.	Data reconciles with source; retries do not create duplicates; missing/stale data is visible.
3. Chatbot & security	Knowledge sync, grounded responses, safe order assistance, audit, WAF integration and threat events.	Tenant-scoped retrieval; validated actions; rate limits and security tests.
4. Marketing intelligence	Selected ad connection, campaign data, order matching, attribution quality, spend/CPA/ROAS reports.	Platform-reported conversions are distinguished from actual orders and profit.
5. AI R&D; Lab	Risk scans, product/pricing/quality analysis, research findings, experiment tracker and outcomes.	Findings have evidence, confidence, missing-data notes, and measurable success criteria.
6. Expansion	Additional connectors, more channels, forecasting, agency workspaces and automation.	Only expand based on pilot feedback and measured value.

Recommended initial implementation order

- Create the project and reproducible local Docker environment.
- Implement authentication, tenancy, policies, and cross-tenant tests before business features.
- Build website registration and secure connector credential management.
- Implement signed event ingestion, idempotency, retries, and source-data reconciliation.

- Deliver basic order/product analytics before introducing AI recommendations.
- Add chatbot and security workflows with explicit guardrails.
- Add R&D; findings only after the underlying metrics are reliable.

9. MVP Acceptance Checklist

- A user can sign in and only access authorized tenant resources.
- A tenant can register a website and securely manage its connector credential.
- A Laravel connector can sync products and orders through a documented contract.
- Repeated event delivery does not create duplicate records.
- Order status changes and employee assignments can be analyzed over time.
- Dashboard metrics disclose time range, currency, freshness, and relevant definitions.
- Chatbot answers are grounded in authorized data and escalates unknown or sensitive cases.
- Security events are recorded safely and mitigation is configurable.
- An R&D; finding shows evidence, confidence, possible causes, missing data, recommended next action, and a way to validate the result.
- Tests cover authentication, authorization, tenant isolation, expired/revoked credentials, signatures, replay, rate limits, and sensitive logging.

10. Monetization & Validation

A possible subscription model is Starter, Growth, and Business. Limits can be based on connected websites, order volume, AI usage, data retention, ad accounts, and advanced R&D capabilities. These are packaging options, not validated prices.

- Run a pilot with a small group of e-commerce businesses before setting final prices.
- Measure whether the product saves time, reduces missed follow-ups, improves visibility into cancellations, or helps merchants run better experiments.
- Track AI cost per tenant and per successful business outcome.
- Keep the starter experience simple; charge more for higher data volume, multi-store reporting, advanced integrations, and experiment tracking.
- Do not claim that AI recommendations guarantee sales growth, profit, security, or product-market fit.

11. Decisions Still to Make

- Confirm the exact first Laravel platform/version and connector installation approach.
- Define website ownership verification and supported connector permissions.
- Finalize canonical order statuses and mapping for each platform/courier.
- Choose initial chatbot LLM/provider, budget controls, and knowledge-update frequency.
- Choose the first ad platform integration and attribution policy.
- Define exact metrics and required cost inputs for contribution-profit reporting.
- Define R&D; report cadence, severity rubric, experiment approval process, and retention policy.
- Confirm production hosting, backups, monitoring, secret management, and incident response.

12. Instructions for the AI Assistant Taking Over This Project

Use this document as the product brief and source of truth for the current intended scope. Do not silently expand the scope or claim unimplemented work is complete.

- First summarize the project understanding and identify contradictions or critical missing decisions.
- Work in small, reviewable phases. Before writing large amounts of code, confirm the framework versions and inspect the actual repository if one exists.
- Produce complete, runnable files when asked, not disconnected snippets presented as a complete application.

- For each phase, list files changed, setup commands, environment variables, migrations, tests, and manual verification steps.
- Write automated tests for tenant isolation, credential security, event idempotency, authorization, and AI guardrails.
- Keep secrets in environment variables or a secret manager; never hardcode production credentials.
- Distinguish proposed design, implemented code, tested behavior, and deployed production capability.
- Do not build the native website builder or landing-page builder unless the product owner explicitly changes scope.
- Prioritize reliable data and security before sophisticated AI; the R&D; Lab depends on trustworthy metrics.

End of brief. This document defines the current product intent and recommended implementation direction. It is a planning handoff, not evidence that the software has already been implemented or deployed.